M1.1: K8s Container OOMKilled (Exit Code 137)

Theory: Linux Kernel OOM Killer selects target processes via '/proc/[pid]/oom_score'. K8s captures Exit Code 137, restarts the container, and tags the Pod status as OOMKilled.

Details: Audits '/var/log/messages' for 'Out of memory: Kill process' indicators. Checks 'kubectl describe pod' for 'Last State: Terminated' with 'Reason: OOMKilled'. Fixes by scaling up 'resources.limits.memory' values.

Trade-off: Overly high limits cause node resource overallocation; excessively low limits lead to endless OOM restart cycles.

M1.2: K8s Pod CrashLoopBackOff Troubleshooting

Theory: K8s schedules exponential restart backoffs when container processes exit with non-zero codes or fail active health checks.

Details: Inspects crash history via 'kubectl logs –previous'. Validates 'ENTRYPOINT' execution permissions, configuration files, and standard exit codes (e.g. 1 for app crash, 127 for binary not found).

Trade-off: Restart backoffs reach up to 5 minutes, leaving services offline if readiness checks are missing.

M1.3: Pod Pending Scheduling Block

Theory: The K8s Scheduler fails to locate any node in the cluster that satisfies the Pod's resource requests or scheduling rules.

Details: Monitors scheduler failure logs via 'kubectl get events'. Adjusts over-provisioned CPU/Memory requests, node taints, tolerations, and affinity configuration mismatches.

Trade-off: Enabling cluster autoscalers automatically provisions cloud VMs but runs the risk of billing spikes.

M1.4: K8s ImagePullBackOff Faults

Theory: The container runtime is unable to pull the target image due to network timeouts, private registry credential failures, or Docker Hub rate limits.

Details: Inspects events via 'kubectl describe pod' for 'ErrImagePull'. Attaches 'imagePullSecrets' to the Pod configuration and sets up local container registries (like Harbor) as pull caches.

Trade-off: Relying on external public hub registries risks rate limiting and single-point registry outages.

M1.5: Ephemeral Storage Overrun Eviction

Theory: Kubelet evicts Pods that exceed their configured local emptyDir or '/tmp' volume storage limit to protect the host node disk.

Details: Declares limits via 'resources.limits.ephemeral-storage'. Implements logrotate configs to direct container output to stdout, and provisions dedicated PVs for large writes.

Trade-off: Strict storage limits enforce clean architecture but can crash legacy systems that rely on large local scratch files.

M1.6: Pod Memory/Disk Pressure Eviction

Theory: Kubelet evicts low-priority Pods when host nodes hit memory availability thresholds (<100Mi) or disk space targets (<10%).

Details: Tracks node states via 'kubectl get nodes' for 'MemoryPressure' or 'DiskPressure'. Eviction priority is resolved using Pod QoS classes: Guaranteed > Burstable > BestEffort.

Trade-off: Guaranteed QoS classes prevent eviction but lock up memory allocation limits, reducing host density.

M2.1: CoreDNS Delay & ndots search Redundancy

Theory: K8s container DNS default 'ndots:5' settings append '.svc.cluster.local' suffixes to external domain lookups, overloading CoreDNS with recursive searches.

Details: Capture lookup sequences via 'tcpdump -i any port 53'. Configure container '/etc/resolv.conf' to use 'ndots:1' or append absolute dots (e.g., 'google.com.') to external queries.

Trade-off: Restricting ndots prevents short-name lookups for services in other namespaces, forcing the use of FQDNs.

M2.2: TCP SYN Flood & Backlog Overflow

Theory: Rapid SYN packet floods fill the connection queues (SYN Queue/Accept Queue), leading to dropped TCP handshakes.

Details: Tracks drop stats via 'netstat -s'. Enables 'net.ipv4.tcp_syncookies = 1' and increases queue sizes via 'net.ipv4.tcp_max_syn_backlog' and 'net.core.somaxconn' sysctl parameters.

Trade-off: Backlog adjustments protect against minor spikes, but massive DDoS attacks require network-layer edge scrubbing.

M2.3: Connection Reset by Peer (RST)

Theory: TCP endpoints transmit RST flags when receiving packets that conflict with current states, or when conntrack tables drop tracking records.

Details: Captures RST traffic via 'tcpdump'. Audits conntrack logs via '/proc/sys/net/netfilter/nf_conntrack_max' and optimizes connection timeout values.

Trade-off: Handling RST packet drops requires robust exponential backoff retry logic on clients.

M2.4: Nginx 502 Bad Gateway Troubleshooting

Theory: Nginx counter-proxy cannot connect to upstream application servers (e.g., Spring Boot, PHP-FPM) because connections are refused or immediately terminated.

Details: Reviews 'error.log' for 'connect() failed (111: Connection refused)'. Validates upstream application health, socket file permissions, and process limits.

Trade-off: Immediate 502 failures require fast custom static fallback HTML pages to protect user experiences.

M2.5: Nginx 504 Gateway Timeout Troubleshooting

Theory: Nginx successfully connects to upstreams but terminates requests when the application fails to return bytes within configured proxy timeout bounds.

Details: Monitors logs for 'upstream timed out (110: Connection timed out)'. Optimizes 'proxy_read_timeout' and 'fastcgi_read_timeout' values, and tunes slow database queries.

Trade-off: Extending timeout bounds ties up Nginx connection pools, exposing gateways to cascading pool exhaustion.

M2.6: Cilium eBPF Socket Redirect Failure

Theory: Cilium employs SOCKMAP BPF maps to redirect packets. Changes in NetworkPolicies or kernel map errors drop packet lookups.

Details: Audits dropped packets using 'cilium monitor –type drop'. Inspects active map entries via 'bpftool map dump' and checks communication to the Cilium Agent.

Trade-off: Disabling eBPF redirect falls back to slower veth-pair routing, degrading network throughput.

M3.1: Terraform State Lock Conflict

Theory: Terraform locks states (via S3 or Consul) during writes to prevent conflicts. Interrupted runs leave stale locks that block all subsequent operations.

Details: Identifies locks via 'Error acquiring the state lock' messages. Releases stale locks via 'terraform force-unlock <Lock-ID>' and validates changes using 'terraform plan'.

Trade-off: Force-unlocking risks state corruption if another runner is actively writing to the backend.

M3.2: PV/PVC Mount Stuck & CSI Lock

Theory: During Pod rescheduling, shared storage volumes (like AWS EBS or Ceph) fail to detach from old nodes due to CSI driver hangs or lock deadlocks.

Details: Audits 'kubectl describe pod' for 'Multi-Attach error'. Validates cloud volume detach state and restarts CSI Controller pods if needed.

Trade-off: Manually detaching volumes unlocks Pods but risks file system corruption if writes are in-flight.

M3.3: Cloud Metadata Service (IMDSv2) Hop Limit

Theory: Cloud host metadata endpoints use local IPs. IMDSv2 requires token handshake headers and locks route hop limits (TTL) to block SSRF vulnerabilities.

Details: Captures 401 response errors on metadata requests. Implements PUT calls for 'X-aws-ec2-metadata-token' and configures hop limits to 2 ('http-put-response-hop-limit=2') for bridged containers.

Trade-off: Enforcing IMDSv2 breaks legacy containers that cannot handle token handshakes.

M3.4: VPC Peering Route Collision

Theory: Routing peers with overlapping CIDR blocks or incomplete route tables cause packet losses or asymmetric routing.

Details: Resolves routing tables for CIDR grid conflicts. Traces path validity using AWS Reachability Analyzer and checks drops via MTR.

Trade-off: Overlapping CIDRs require NAT workarounds; clean global IPAM planning is the only permanent solution.

M3.5: Vault Dynamic Lease Expiration

Theory: Vault dynamic backends issue database/cloud credentials bound to Lease TTLs. If client renewals fail, Vault revokes and deletes the credentials.

Details: Checks Vault logs for 'lease revoked' entries. Verifies client SDK auto-renew helper loops and implements fallback long-lived user profiles for emergencies.

Trade-off: Short lease TTLs limit breach exposure but increase dependency on Vault availability.

M3.6: Cloud Storage Bucket Policy Over-exposure

Theory: Storage buckets configured with wildcard policies ('*') allow anonymous internet users to read or write sensitive objects.

Details: Scans policies for 'Effect: Allow, Principal: *' configurations. Enforces central "Block Public Access" switches and issues signed URLs with short expirations.

Trade-off: Restricting public access blocks standard CDN integrations, requiring segregated public and private storage buckets.

M4.1: Prometheus TSDB High Cardinality Churn

Theory: High-cardinality labels (like user IDs or port numbers) generate millions of transient time series, overloading Prometheus TSDB memory.

Details: Checks series stats via '/api/v1/status/tsdb'. Implements Prometheus metric relabeling configurations to drop or replace high-cardinality tags.

Trade-off: Dropping metrics labels reduces query granularity but protects Prometheus servers from crashing.

M4.2: Log Collector Backpressure & Queue Saturation

Theory: Slow Elasticsearch write paths trigger Fluentd memory queue saturation, prompting client backpressure that stalls upstream apps.

Details: Configures Logstash persistent disk queues ('queue.type: persisted'). Fixes ES write throttling issues and increases bulk API batch sizes.

Trade-off: Local disk queues protect against brief spikes but eventually fill up if upstream traffic persistently exceeds database ingress capacity.

M4.3: Distributed Tracing Context Propagation Loss

Theory: Microservices fail to parse or inject W3C ‘traceparent‘ headers during RPC calls or async message passes, breaking trace chains.

Details: Checks if reverse proxies strip the ‘traceparent‘ header. Wraps asynchronous ThreadPool runnables with TraceContext propagation wrappers.

Trade-off: Logging all trace contexts increases CPU usage; high-throughput systems require sampling limit tunings.

M4.4: Grafana Large Range Slow Query Optimization

Theory: Querying long time spans (e.g. 30 days) triggers full TSDB database sweeps, consuming massive RAM and slowing dashboard rendering.

Details: Configures Grafana query variables with dynamic ‘$__interval‘ steps. Deploys Prometheus Recording Rules to pre-aggregate high-resolution metrics.

Trade-off: Recording rules discard raw, second-level historical metrics detail in favor of speed.

M4.5: Alertmanager Flapping, Grouping & Deduplication

Theory: Mismatching alert configurations flood administrators with duplicate alerts or trigger flapping cycles on flapping states.

Details: Optimizes ‘group_by‘, ‘group_wait‘ (e.g. 30s to bundle events), ‘group_interval‘, and increases ‘repeat_interval‘ to 12h for minor alerts.

Trade-off: High group wait times delay notifications for critical production outages.

M5.1: SRE SLI/SLO Error Budget Burn Rate

Theory: SRE burn rates measure how fast a service is consuming its error budget, triggering alerts before SLO thresholds are crossed.

Details: Writes PromQL alerting rules based on multi-window multi-burn-rate strategies (e.g., 1h/6h burn rate $>$ 14.4 to trigger page alerts).

Trade-off: Multi-window rules are complex to configure and require fine-tuned metric resolutions.

M5.2: Chaos Engineering Network Loss & CPU Stress

Theory: Injecting controlled network delay and CPU stress validates application self-healing, routing redundancy, and HPA performance.

Details: deploys Chaos Mesh. Uses Linux Traffic Control (tc) to inject 10% packet drop and uses ‘stress-ng‘ to trigger HPA scaling rules.

Trade-off: Chaos runs risk cascading failures; blast-radius safety switches must be implemented.

M5.3: Graceful Service Degradation & Feature Flags

Theory: Disabling non-core features (like reviews or recommendations) via feature flags frees up resources for core transactional workflows during traffic surges.

Details: Integrates feature flags (like LaunchDarkly) and deploys circuit breakers with fallback static objects to dodge remote API overheads.

Trade-off: Degrading services impacts user experiences but prevents catastrophic site outages.

M5.4: Database Seconds-level PITR Replay Validation

Theory: Recovering database states from physical corruption requires replaying Write-Ahead Logs (WAL) up to a precise second.

Details: Validates Postgres pg_backrest backups. Declares ‘restore_command‘ and configures ‘recovery_target_time‘ to spin up staging instances.

Trade-off: Replaying WAL logs is IO-heavy and takes time, requiring automated staging test validation.

M5.5: SRE Blameless Post-Mortem Standards

Theory: Distrusting personal error is useless. Focuses on system architecture and operational workflows to identify root causes and prevent repeat failures.

Details: Maps incident timelines (Trigger $\rightarrow$ Alert $\rightarrow$ Investigate $\rightarrow$ Mitigate). Employs the ”5 Whys” method to assign tracked Action Items with owners and due dates.

Trade-off: Post-mortems require organizational trust; blame-shifting cultures lead to hidden errors.

Zone T1: SRE Firefighting CLI & Configs

kubectl logs, CoreDNS resolv.conf, somaxconn backlog, CSI Attach/Detach, IMDSv2 Hop Limit, PromQL Burn Rate, Chaos Mesh

Zone T2: System Faults & Diagnostics

OOMKilled_137_memory_saturation, CrashLoopBackOff_entrypoint_permission, CoreDNS_ndots_redundant_query, Nginx_502_504_gateway_timeout, CSI_volume_multi_attach_lock, Vault_lease_revocation_token, Prometheus_TSDB_series_cardinality, SRE_error_budget_blameless_post_mortem